Clustering y K-Means

Aprendizaje No Supervisado · Agrupamiento · Perfiles de Oyentes de Spotify

Inteligencia Artificial · Sexto Semestre · Ingeniería de Sistemas de Información · PUCE

Objetivos de Aprendizaje

Al finalizar esta clase serás capaz de:

1. Explicar qué es el clustering y en qué se diferencia del aprendizaje supervisado y de PCA.
2. Describir el algoritmo K-Means paso a paso y su función objetivo: la inercia (WCSS).
3. Elegir K con el método del codo y el coeficiente de Silhouette, e interpretar los centroides.
4. Entrenar el modelo final con el 100% de los datos, exportarlo con joblib y asignar oyentes nuevos.

Clustering: la Otra Familia No Supervisada

Sin variable objetivo y: el algoritmo solo recibe X

Reducción de dimensionalidad (clase anterior)

- Resume muchas **variables** (columnas) en pocas.
- Algoritmo visto: PCA.
- Ejemplo: 10 variables de audio → mapa 2D de canciones.

Clustering (hoy)

- Agrupa **observaciones** (filas) parecidas.
- Algoritmo de hoy: K-Means.
- Ejemplo: 1 200 oyentes → 3 perfiles de escucha.

Aspecto	Supervisado (clases anteriores)	Clustering (hoy)
¿Tiene etiqueta y?	Sí: aprobado, género, riesgo…	No: no existe
¿Qué aprende?	Una función f(X) → y	Grupos naturales dentro de X
¿Cómo se evalúa?	Accuracy, F1, RMSE	Inercia, Silhouette, sentido de negocio

¿Qué es el Clustering?

Clustering = agrupamiento

Definición: técnica de aprendizaje no supervisado que divide un conjunto de observaciones en grupos (clusters), de modo que las observaciones de un mismo grupo sean más parecidas entre sí que a las de otros grupos.

En lenguaje simple: el algoritmo recibe datos sin etiquetas y responde «¿qué grupos naturales hay aquí?».

Cohesión (dentro del grupo)	Separación (entre grupos)
• Los puntos de un cluster están **cerca** entre sí. • Se mide con la inercia (WCSS).	• Los clusters están **lejos** unos de otros. • Se mide con el coeficiente de Silhouette.

No hay respuesta correcta predefinida: el nombre de cada grupo lo pone el analista

¿Dónde se Aplica el Clustering?

Casos típicos donde no existe una etiqueta previa

Segmentación de clientes: Un banco agrupa clientes por comportamiento de gasto para diseñar productos.

Contenido y streaming: Una plataforma agrupa oyentes o canciones similares para recomendar.

Salud: Se buscan subtipos de pacientes con perfiles clínicos parecidos.

Territorio: Se agrupan cantones de Ecuador por indicadores socioeconómicos para priorizar políticas.

Origen: Stuart Lloyd describió el algoritmo en 1957 (publicado en 1982); James MacQueen lo llamó «k-means» en 1967.

Fuentes: Lloyd (1982), IEEE Trans. Information Theory · MacQueen (1967), Proc. 5th Berkeley Symposium.

Dataset — 1 200 Oyentes de Spotify Ecuador

oyentes_spotify_ecuador_A.csv · sintético · sin nulos · sin columna de etiqueta

Variable	Rango en los datos	Qué mide
`oyente_id`	EC0001 a EC1200	Identificador: no entra en el modelo
`horas_diarias`	0.50 a 7.55	Horas promedio de escucha por día
`diversidad`	1 a 12	Géneros distintos escuchados en el mes
`energia_promedio`	0.25 a 0.98	Energía de la música (0 = tranquila, 1 = muy energética)
`skips_por_hora`	0.10 a 5.64	Canciones saltadas por hora de escucha

⚠️ **Sin etiqueta:** ninguna columna dice qué tipo de oyente es cada persona. K-Means debe descubrir los perfiles solo con las 4 variables.

Los datos son simulados con fines didácticos; no provienen de Spotify.

Cómo Funciona K-Means: Paso a Paso

Algoritmo iterativo: mueve K centroides hasta que los grupos se estabilizan

1. **Elegir K:** el usuario define el número de grupos (ej. K = 3 perfiles).
2. **Inicializar:** se colocan K centroides iniciales; scikit-learn usa **k-means++**, que los elige separados entre sí.
3. **Asignar:** cada punto va al centroide más cercano (distancia euclidiana).
4. **Actualizar:** cada centroide se mueve al **promedio** de los puntos de su cluster.
5. **Repetir 3 y 4** hasta que ningún punto cambie de cluster (convergencia).

Asignar y Actualizar: las Dos Fórmulas del Ciclo

$x^{(i)}$ = oyente i · μ_k = centroide del cluster k · C_k = puntos del cluster k

Paso 3 — Asignar

$$c^{(i)} = \arg\min_k \ ||x^{(i)} - \mu_k||^2$$

Lectura

- Para cada oyente se calcula la distancia a los K centroides.
- **argmin** = el k que da la distancia más pequeña.
- Ese k es su cluster.

Paso 4 — Actualizar

$$\mu_k = \frac{1}{|C_k|} \sum_{x \in C_k} x$$

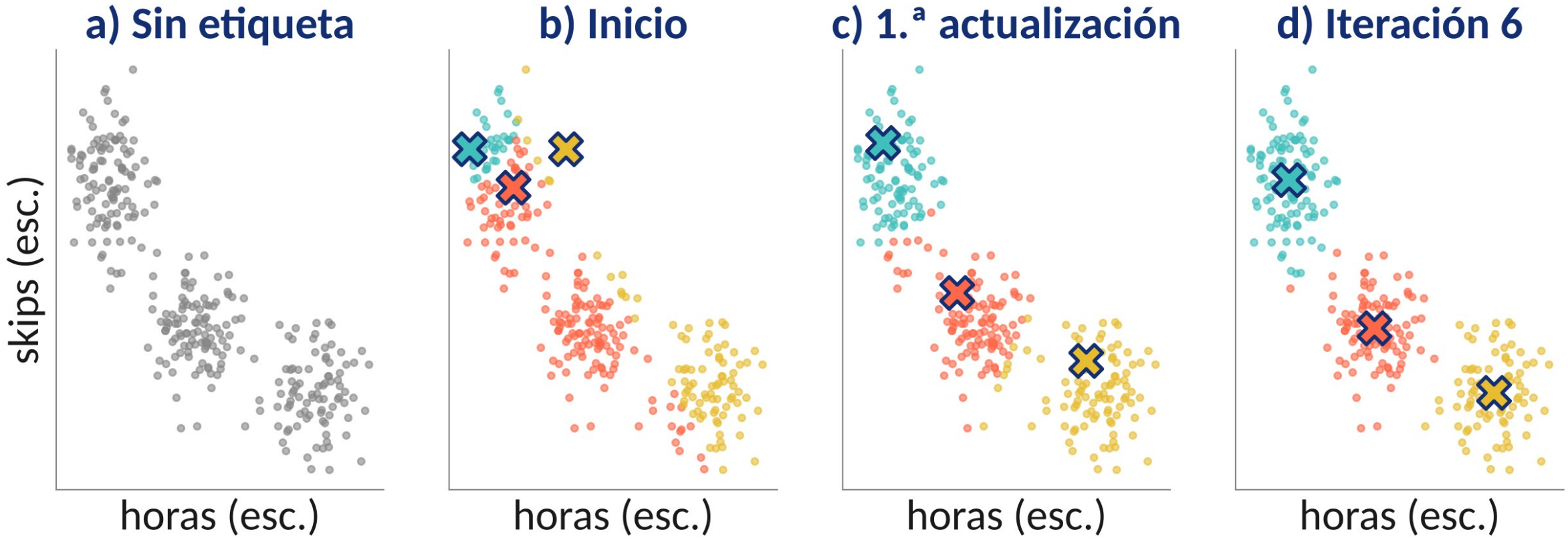
Lectura

- $|C_k|$ = cuántos puntos tiene el cluster k.
- El nuevo centroide es el **promedio** de esos puntos.
- El centroide «sigue» a sus puntos.

Cada ciclo asignar → actualizar nunca aumenta la inercia: solo baja o se mantiene

Visualización de las Iteraciones (K = 3)

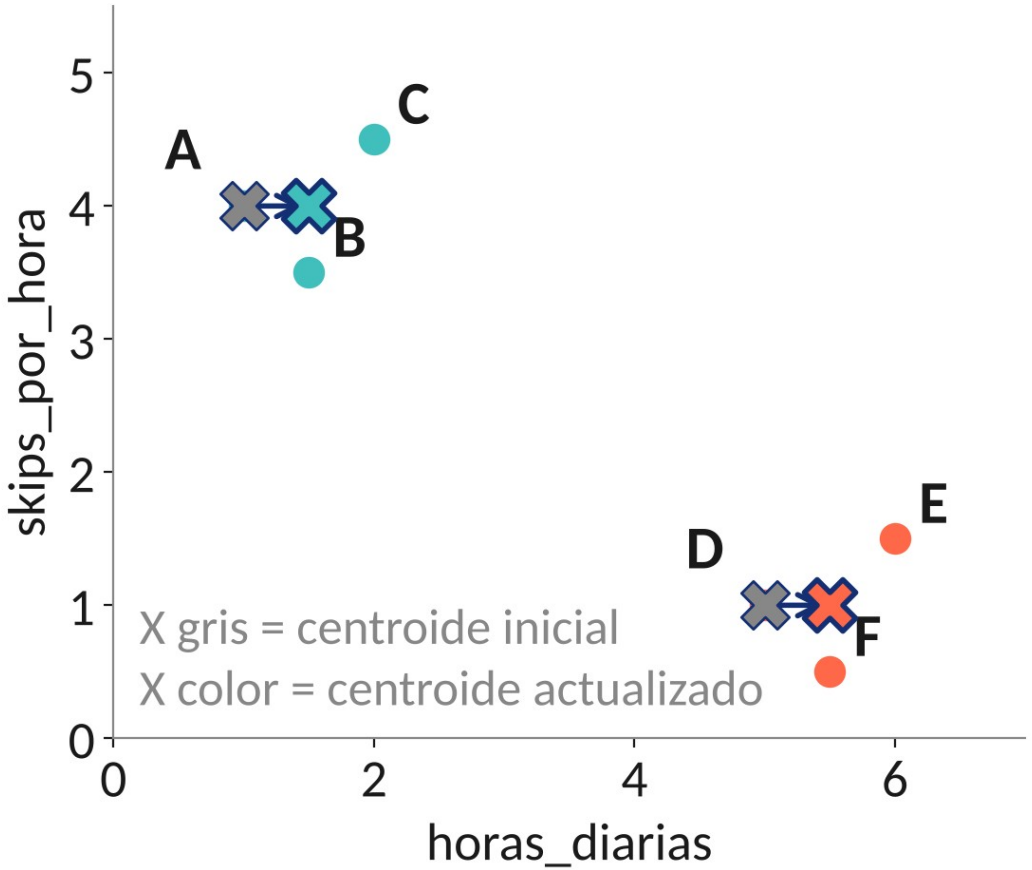
Datos reales: 300 oyentes al azar, 2 variables escaladas, inicio deliberadamente malo



- **b)** Los 3 centroides iniciales empiezan juntos en una esquina: la primera asignación es mala.
- **c)** Tras actualizar, cada centroide se mueve al promedio de sus puntos y los grupos mejoran.
- **d)** En la iteración 6 ningún punto cambia de cluster: el algoritmo converge.

Ejemplo a Mano: una Iteración (ilustrativo)

6 oyentes, 2 variables, K = 2 · centroides iniciales: c1 = A y c2 = D



Oyente	(horas, skips)	d a c1	d a c2	Cluster
A	(1.0, 4.0)	0.00	5.00	1
B	(1.5, 3.5)	0.71	4.30	1
C	(2.0, 4.5)	1.12	4.61	1
D	(5.0, 1.0)	5.00	0.00	2
E	(6.0, 1.5)	5.59	1.12	2
F	(5.5, 0.5)	5.70	0.71	2

- **Asignar:** cada oyente va al centroide con la distancia menor.
- **Actualizar:** c1 = promedio de A, B, C = (1.5, 4.0); c2 = promedio de D, E, F = (5.5, 1.0).
- Otra iteración no cambia nada: **converge**.

Distancia Euclidiana: la Medida de Parecido

La misma distancia que medirías con una regla, extendida a p variables

Con 2 variables

$$d(A, B) = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$$

Con p variables (aquí p = 4)

$$d(A, B) = \sqrt{\sum_{j=1}^{p} (b_j - a_j)^2}$$

Ejemplo (oyentes A y C del ejemplo anterior)

- A = (1.0, 4.0) y C = (2.0, 4.5).
- d = √((2.0 − 1.0)² + (4.5 − 4.0)²)
- d = √(1 + 0.25) = **1.12**

Rango y lectura

- Rango: 0 a ∞.
- 0 = dos oyentes idénticos.
- Más pequeña = más parecidos.

⚠️ **Ojo:** la distancia suma diferencias de variables con unidades distintas: por eso hay que escalar (ver más adelante).

La Función Objetivo: Inercia (WCSS)

WCSS = Within-Cluster Sum of Squares (suma de cuadrados dentro de los clusters)

$$\text{WCSS} = \sum_{k=1}^{K} \sum_{x \in C_k} \|x - \mu_k\|^2$$

Rango: 0 a ∞ → más bajo = clusters más compactos

¿Qué mide?

- Suma la distancia al cuadrado de cada punto a su centroide.
- En scikit-learn: atributo `inertia_`.

¿Por qué al cuadrado?

- Penaliza más los puntos muy lejanos.
- K-Means busca los centroides que la minimizan.

⚠️ **Limitación:** la inercia siempre baja al aumentar K; con un cluster por punto vale 0. Por eso no sirve sola para elegir K.

Antes de K-Means: Escalar las Variables

Ejemplo con desviaciones estándar reales del dataset (ilustrativo)

StandardScaler

$$z = \frac{x - \mu}{\sigma}$$

media 0 · desviación 1

Diferencia entre 2 oyentes	Sin escalar	Escalada (÷ σ)
diversidad: 3 géneros (σ = 2.74)	3.00	1.09
energia_promedio: 0.40 (σ = 0.16)	0.40	2.50

Sin escalar

- La diversidad domina la distancia solo porque sus números son más grandes.
- Un salto de 0.40 en energía casi no cuenta.

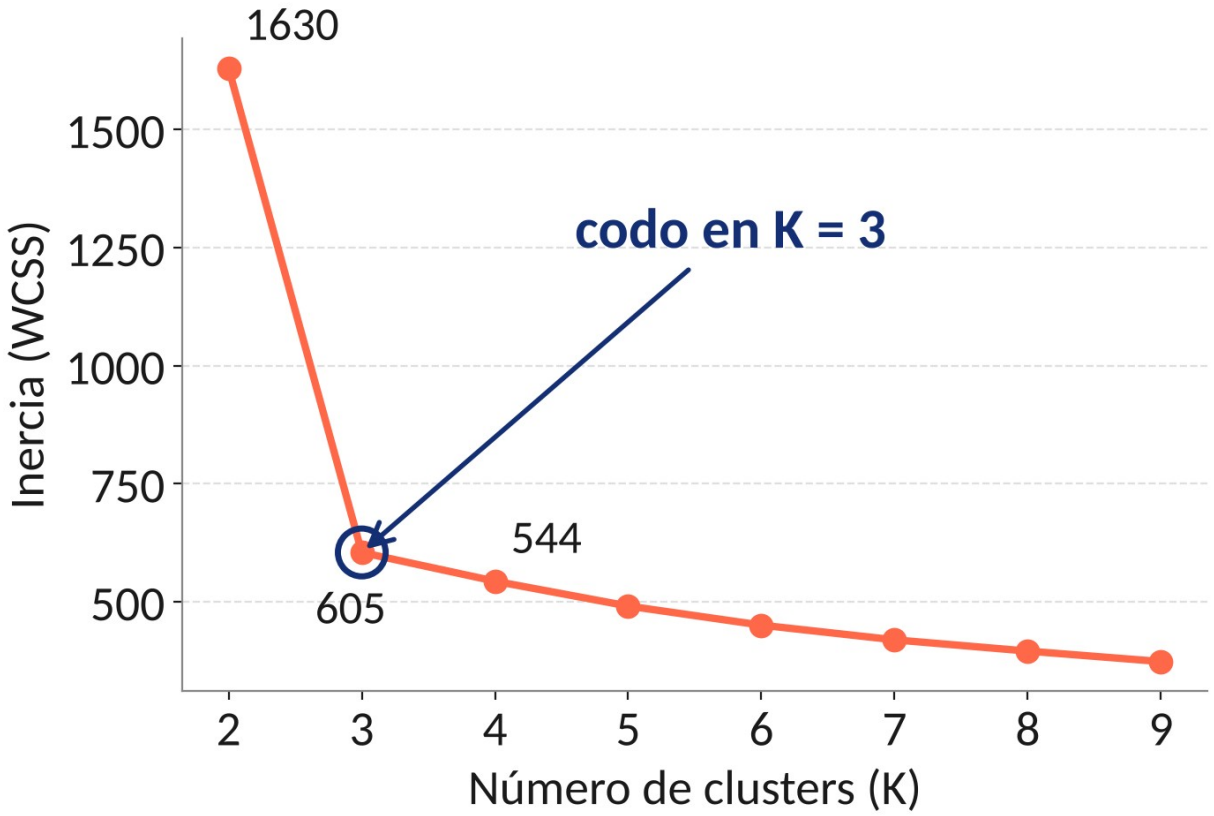
Escalado

- Cada diferencia se mide en desviaciones estándar.
- Ahora el salto de energía (2.5 σ) pesa más que el de diversidad (1.1 σ).

Regla: siempre StandardScaler. En estos oyentes el resultado casi no cambia porque los grupos están muy separados; en otros datos cambia por completo (taller, Bloque B5).

Método del Codo (Elbow Method)

Inercia real de K = 2 a 9 · oyentes escalados



Rango: 0 a ∞ · se busca el codo, no el valor más bajo

Cómo se aplica

- Entrenar K-Means con K = 2, 3, …, 9.
- Guardar `inertia_` de cada modelo.
- Graficar K vs. inercia.
- El «codo» es donde la mejora se vuelve pequeña.

Lectura

- K = 2 → 3: baja 1 025.
- K = 3 → 4: baja solo 61.
- Codo en **K = 3**.

Coeficiente de Silhouette

Mide cohesión y separación a la vez, para cada punto i

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\ b(i)\}}$$

Rango: −1 a 1 → cercano a 1 = deseado

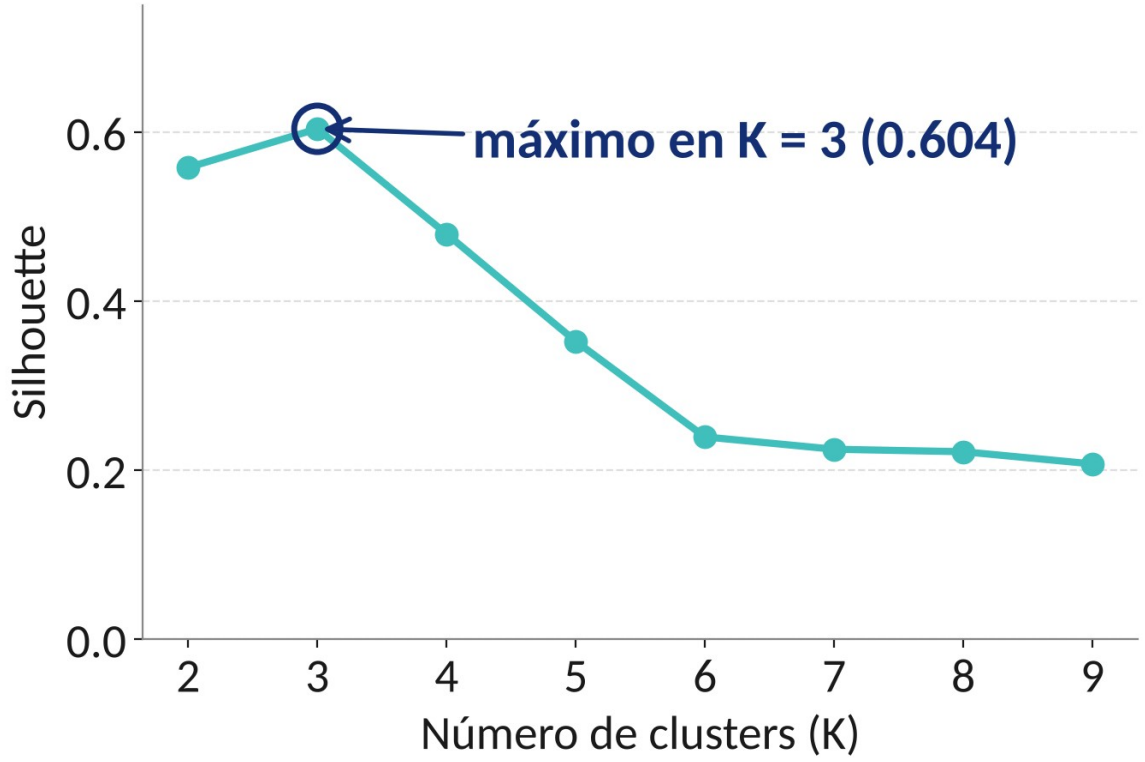
- **a(i)** = distancia media a **su** cluster.
- **b(i)** = distancia media al cluster vecino **más cercano**.
- Score del modelo = promedio de s(i).

Silhouette	Interpretación	Qué indica
0.71 a 1.00	**Excelente**	Estructura fuerte: clusters muy separados
0.51 a 0.70	**Bueno**	Estructura razonable, poco solapamiento
0.26 a 0.50	**Débil**	Separación poco clara: revisar K
≤ 0.25	**Sin estructura**	Los clusters no son significativos

Fuentes: Rousseeuw (1987), J. Comput. Appl. Math. · Kaufman y Rousseeuw (1990), Finding Groups in Data.

Silhouette vs. K: Criterio Combinado

Silhouette real de K = 2 a 9 · requiere K ≥ 2



Métrica	K elegido
Inercia (codo)	K = 3
Silhouette (máx.)	K = 3 (0.604)

Lectura

- 0.604 está en el rango **bueno** (0.51 a 0.70).
- K = 4 baja a 0.480 (débil).
- Ambas métricas coinciden.

Cuando el codo y el máximo de Silhouette coinciden, la elección de K es sólida

La Clase KMeans() (1/2): Parámetros

Se pasan al crear el objeto, antes de fit()

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, n_init=10, random_state=42)
```

Parámetro	Defecto	Qué controla
`n_clusters`	8	Número de grupos K
`init`	'k-means++'	Cómo elegir los centroides iniciales
`n_init`	'auto' (1 vez)	Repeticiones con otro inicio; gana la menor inercia
`max_iter`	300	Máximo de iteraciones asignar → actualizar
`random_state`	None	Semilla: mismo resultado en cada ejecución

⚠️ **Ojo:** según el inicio, K-Means puede quedarse en una solución que no es la mejor posible; por eso se repite con n_init=10.

La Clase KMeans() (2/2): Métodos y Atributos

K-Means no recibe y: fit() solo usa X

Métodos	Atributos (solo existen después de fit)
• `fit(X)`: entrena (busca los centroides). • `predict(X)`: asigna cada fila al centroide más cercano. • `fit_predict(X)`: fit + predict en una sola llamada.	• `labels_`: cluster de cada fila. • `cluster_centers_`: centroides, en la escala de los datos con que se entrenó. • `inertia_`: inercia (WCSS) final.

⚠️ **No son parámetros:** `labels_`, `cluster_centers_` e `inertia_` no se pasan al crear el modelo; terminan en guion bajo porque se calculan al entrenar.

⚠️ **Números sin significado:** los clusters salen como 0, 1, 2. El número no indica orden ni calidad; con otra semilla pueden cambiar de número.

Flujo Completo (1/3): Datos, X y Escalado

Construir X solo con las 4 variables numéricas

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('oyentes_spotify_ecuador_A.csv')
X = df.drop(columns=['oyente_id'])   # 4 variables

escalador = StandardScaler()
X_esc = escalador.fit_transform(X)    # media 0, desv. 1
```

- `oyente_id` es un identificador: no describe el comportamiento, por eso se quita.
- `X_esc` se usa solo para **explorar** qué K conviene (siguiente diapositiva: Flujo 2/3).
- No hay división train/test: no existe una `y` que predecir.

Flujo Completo (2/3): Elegir K

Un bucle entrena un modelo por cada K y guarda las dos métricas

```python
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

for k in range(2, 10):                          # K = 2 a 9
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    etiquetas = km.fit_predict(X_esc)  # cluster de cada fila
    sil = silhouette_score(X_esc, etiquetas)
    print(k, round(km.inertia_, 1), round(sil, 3))
```

K	2	3	4	5	6
Inercia	1 630.1	**604.6**	543.8	491.2	450.7
Silhouette	0.558	**0.604**	0.480	0.353	0.240

Flujo Completo (3/3): Pipeline y Centroides

Pipeline (visto en clases anteriores): escalar + KMeans en un solo objeto

```
from sklearn.pipeline import Pipeline

modelo = Pipeline([('escalar', StandardScaler()),
                   ('kmeans', KMeans(n_clusters=3, n_init=10,
                                     random_state=42))])
df['cluster'] = modelo.fit_predict(X)    # X sin escalar

columnas = list(X.columns)
print(df.groupby('cluster')[columnas].mean().round(2))
```

- El Pipeline escala solo: se le pasa `X` original, no `X_esc`.
- `groupby().mean()` da los centroides en **escala original** (horas, géneros...), fáciles de interpretar.

Resultado: 3 Perfiles de Oyente

Centroides reales en escala original · Silhouette = 0.604 · Inercia = 604.6

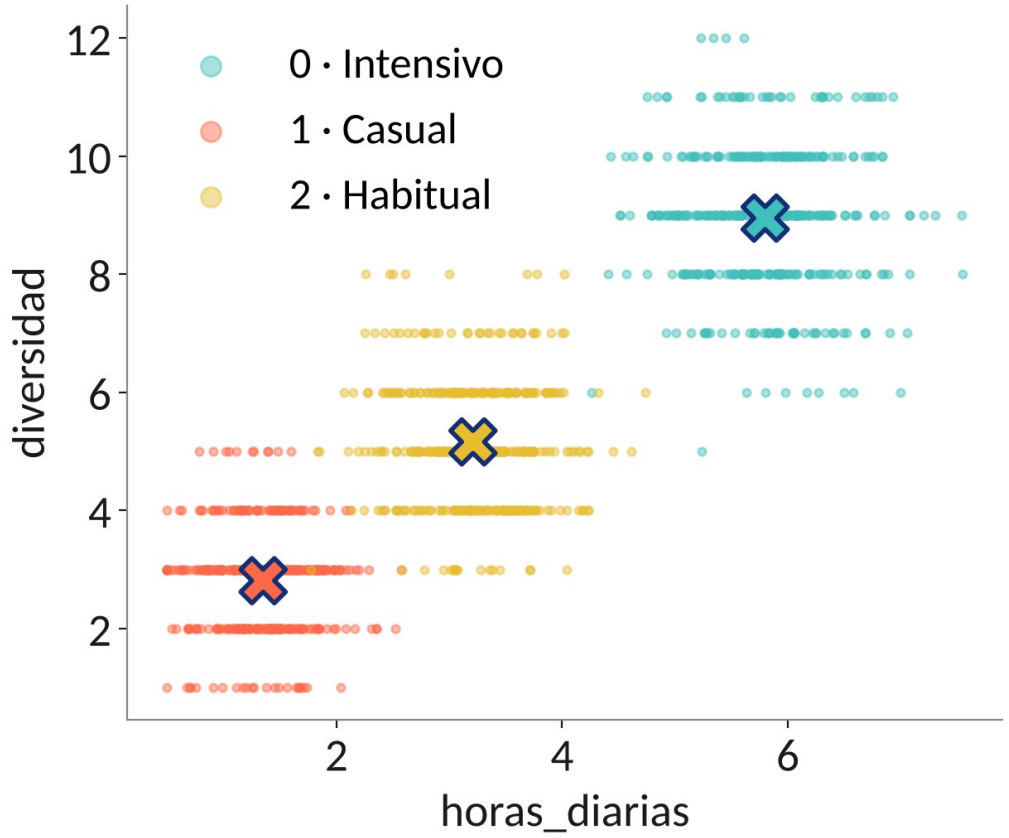
Cluster	Oyentes	horas_diarias	diversidad	energia	skips/hora	Perfil (analista)
0	400	5.80	8.95	0.82	1.07	Intensivo
1	399	1.35	2.81	0.45	3.83	Casual
2	401	3.21	5.16	0.62	1.89	Habitual

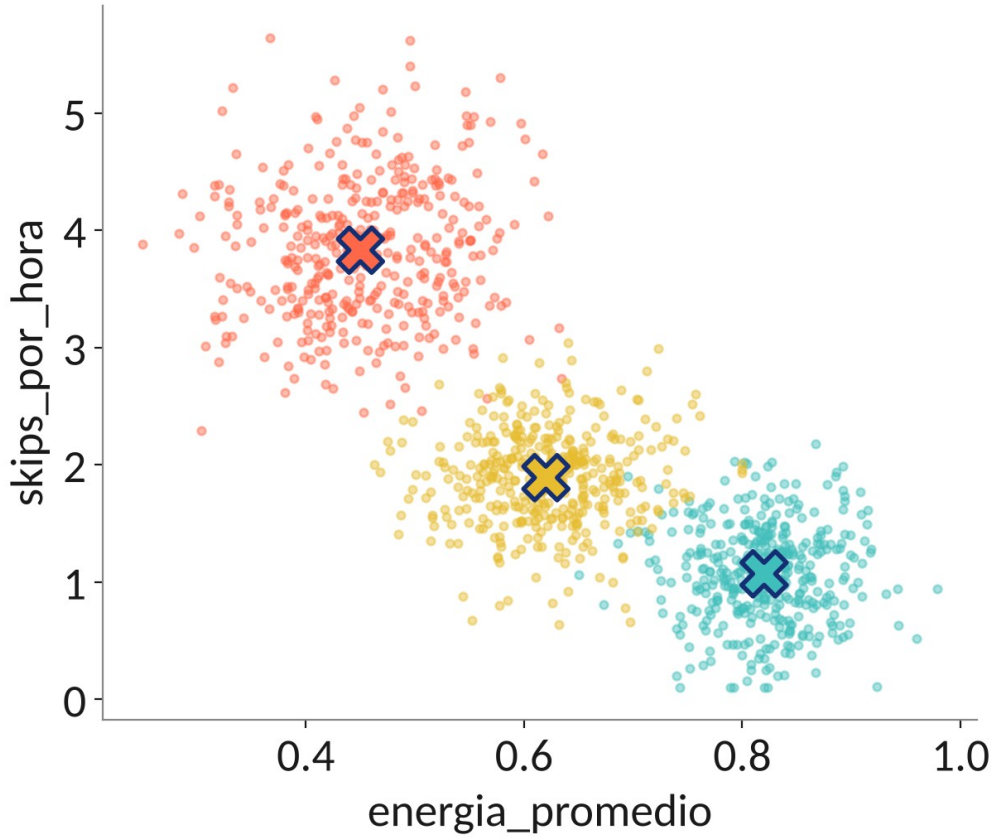
- **Intensivo:** muchas horas, muchos géneros, música energética y casi no salta canciones.
- **Casual:** pocas horas, pocos géneros y salta muchas canciones.
- **Habitual:** perfil intermedio y estable.

⚠️ **Los nombres los pone el analista:** K-Means solo entrega 0, 1 y 2. El nombre sale de leer los centroides con criterio de negocio.

Mapa de los Clusters

1 200 oyentes coloreados por cluster · X = centroide





• Los 3 grupos se separan con claridad; solo se tocan en los bordes (de ahí un Silhouette bueno, no excelente).

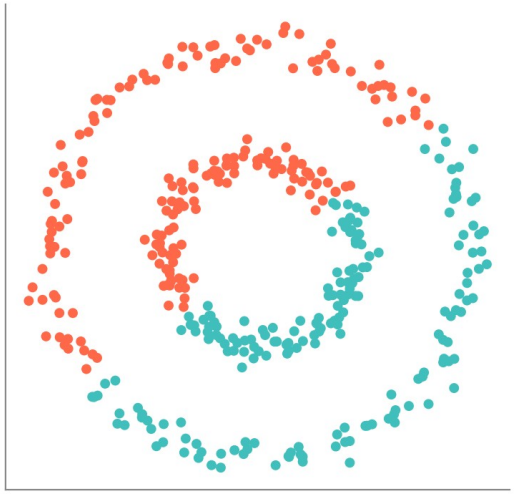
¿Cuándo NO Usar K-Means?

Datos generados con sklearn.datasets (ilustrativo)

Lunas



Anillos



- En ambos casos los grupos reales tienen otra forma.
- K-Means los corta con una línea recta.

Limitaciones

- Supone grupos redondos (esféricos) y de tamaño parecido.
- Los outliers desplazan el promedio del centroide.

Alternativas

- Variables categóricas: K-Modes.
- Formas irregulares o ruido: DBSCAN.
- Sin idea de cuántos grupos: clustering jerárquico.

Entrenamiento Final con el 100% de los Datos

El modelo final se entrena con los 1 200 oyentes y se exporta

Análisis (exploración)

- Bucle de K y `modelo`: sirvieron para elegir K = 3 y nombrar perfiles.

Modelo final (100%)

- `modelo_final`: escalar + KMeans, listo para datos nuevos.

```
modelo_final = Pipeline([
    ('escalar', StandardScaler()),
    ('kmeans', KMeans(n_clusters=3, n_init=10,
                      random_state=42))])
modelo_final.fit(X)                    # 100%: 1 200 oyentes
perfiles = {0: 'Intensivo', 1: 'Casual', 2: 'Habitual'}
```

Observa: sin train/test, el análisis ya usó el 100%. Con la misma semilla los números coinciden; revisa los centroides antes de fijar `perfiles`.

Exportación con joblib (1/2)

Un solo .pkl guarda el Pipeline entrenado y los perfiles: no hace falta escalar a mano ni reentrenar

```
import joblib
joblib.dump({'modelo': modelo_final, 'perfiles': perfiles},
            'modelo_kmeans_spotify.pkl')

# en otro programa (ej. un servicio web):
import joblib
import pandas as pd
paquete = joblib.load('modelo_kmeans_spotify.pkl')
modelo_cargado, perfiles = paquete['modelo'], paquete['perfiles']
```

⚠️ **Versión:** carga el .pkl con la misma versión de scikit-learn con la que se guardó.

Uso con Oyentes Nuevos (2/2)

Mismas 4 columnas que X, con los mismos nombres y sin escalar

```
nuevos = pd.DataFrame({
    'horas_diarias':    [6.2, 1.0, 3.0],
    'diversidad':       [10, 2, 5],
    'energia_promedio': [0.85, 0.40, 0.60],
    'skips_por_hora':   [0.8, 4.5, 2.0]})

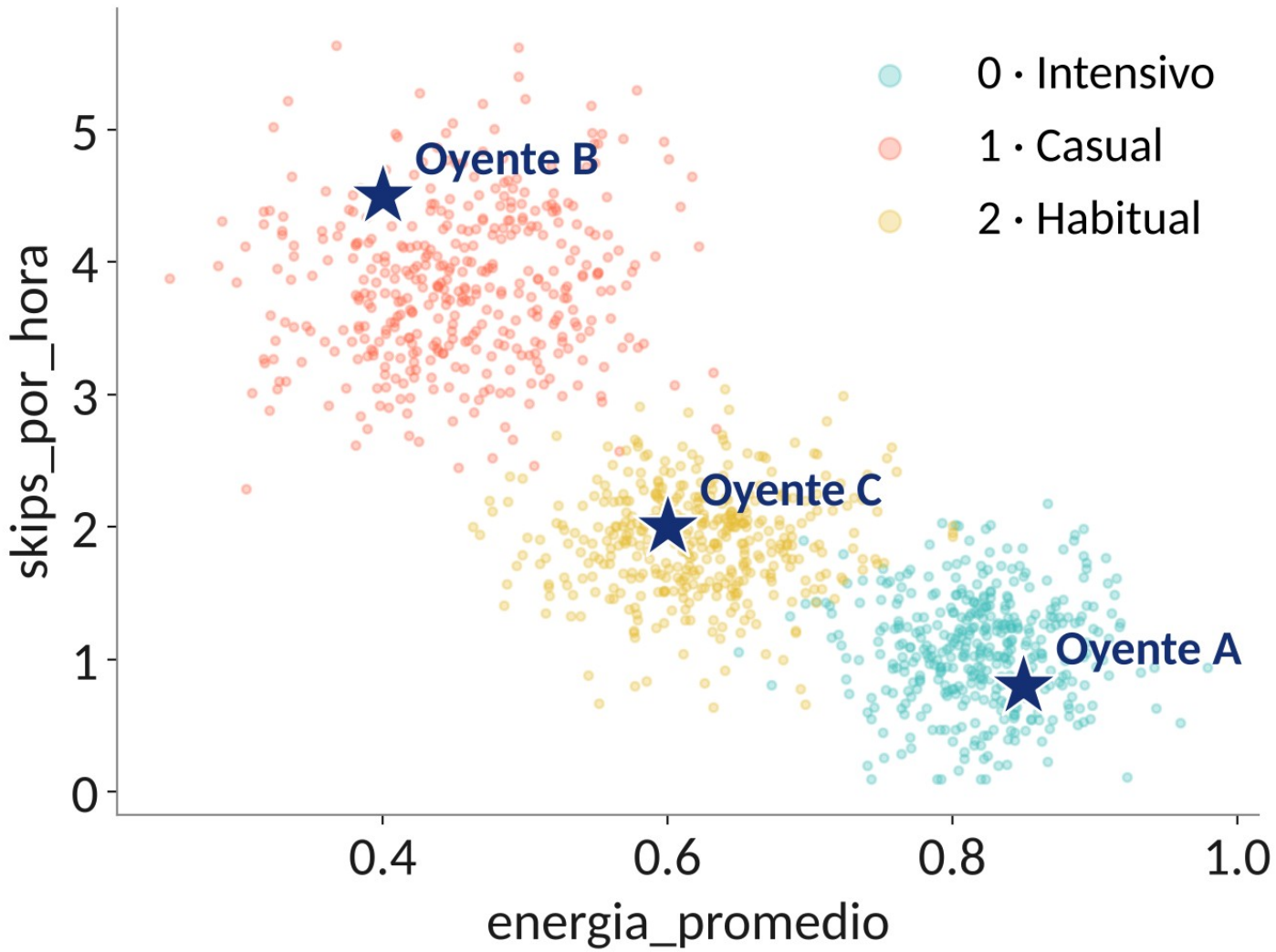
grupos = modelo_cargado.predict(nuevos)   # [0 1 2]
print([perfiles[g] for g in grupos])
```

Oyente A → 0 Intensivo · Oyente B → 1 Casual · Oyente C → 2 Habitual

- `predict()` escala con las medias aprendidas y asigna el centroide más cercano: no reentrena.

Oyentes Nuevos en el Mapa

Estrellas = oyentes A, B y C asignados por el modelo exportado



Lectura

- **A:** mucha energía, pocos skips → Intensivo.
- **B:** poca energía, muchos skips → Casual.
- **C:** valores intermedios → Habitual.
- Uso real: recomendar música según el perfil.